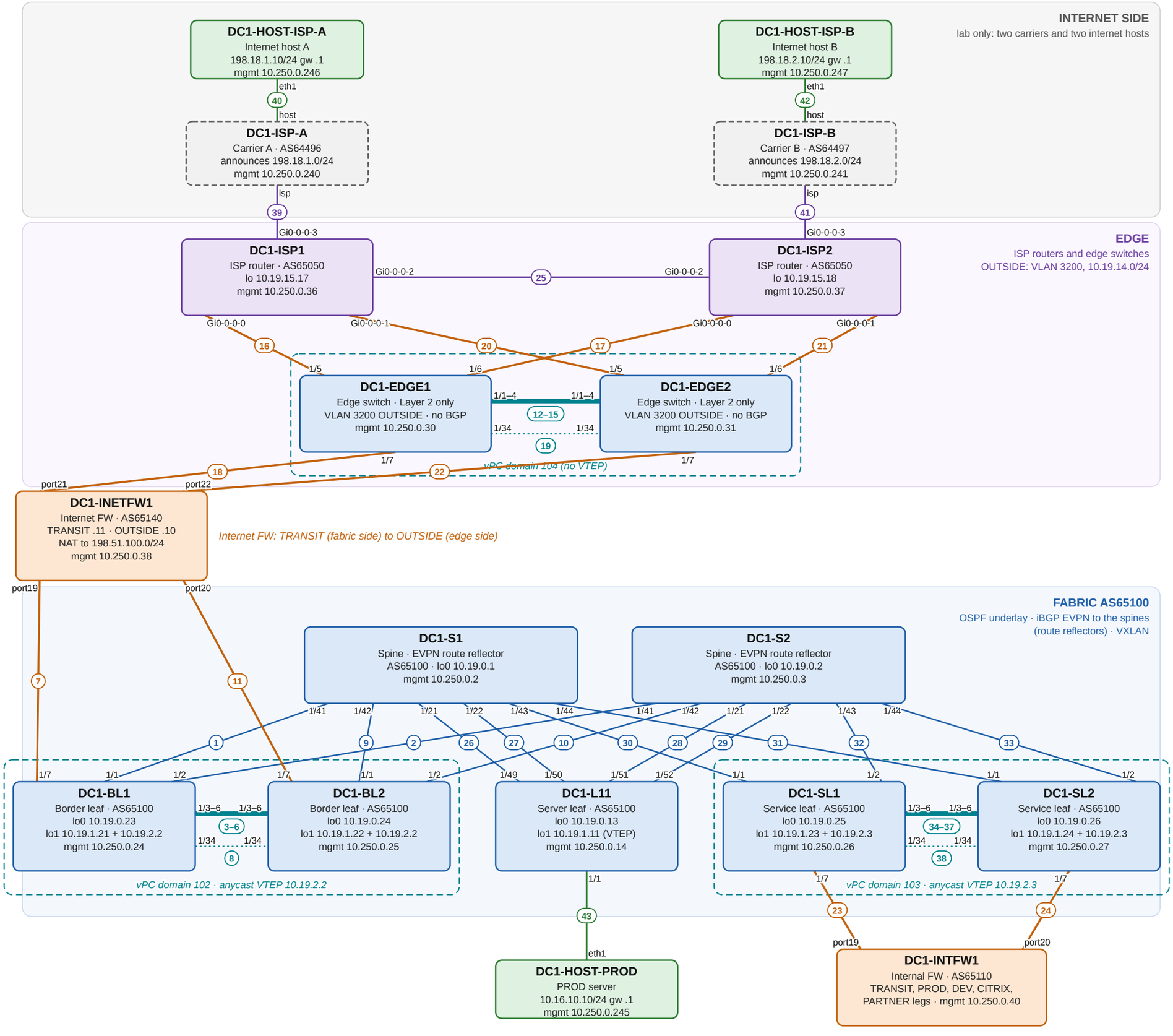
DC1 lab path: every device and cable



Legend

Boxes: what runs in the lab

- Nexus switch: N9Kv 10.6(4)M lite (vrnetlab), 8 GB, 3 vCPU
- FortiGate firewall: FRR 10.6.0 stand-in
- Cisco 8201 ISP router (IOS XR): FRR 10.6.0 stand-in
- Carrier (exists only in the lab): FRR 10.6.0
- Test host: Alpine 3.22

Lines: what each cable is

- Fabric link: routed /31, OSPF UNDERLAY area 0, MTU 9216
- vPC peer-link: port-channel1000, 4 members (drawn as one)
- vPC peer-keepalive: Eth1/34, VRF VPC-KEEPALIVE /31
- LACP bundle, one member per vPC switch: 802.1Q trunk
- Routed link outside the fabric (ISP cross-link, carriers)
- Host link
- vPC pair (domain number, anycast VTEP)
- 26 — Link number: addressing on page 2
- 1/41 — Port at that end of the cable

How to read the ports

Nexus: 1/41 = Ethernet1/41 in the config;
containerlab calls the same port eth41.
FortiGate stand-ins: port19…port22 (FortiOS names).
ISP stand-ins: Gi0-0-0-0 = Gi0/0/0/0 (IOS XR name with / replaced by -, as Linux needs).

The lab at a glance

Lab name vamd-dc1-p3 · 18 nodes · 43 links
Containers: clab-vamd-dc1-p3-<node name>
Mgmt network vamd-dc1-mgmt 10.250.0.0/24
(every box shows its mgmt address)
9 Nexus × 8 GB = 72 GB; whole lab about 80 GB.
Ran on AWS m8i.8xlarge (32 vCPU, 123 GB).
Deploy + push + carve: 27 minutes, no manual step.
Only DC1-L11 of the 20 server leaves is in the lab; only one unit of each firewall/ISP pair (INTFW1, INETFW1) except the ISP routers (both).
Spine configs still list all 26 EVPN clients: the sessions to absent devices stay down (expected).

Not in this lab (production DC1 has them)

L1–L10, L12–L20, BGW1/2, DCI1/2, DMZ1/2, INTFW2, INETFW2, SDWANFW1/2, VPNFW1/2, CLOUD1/2. Their SVIs exist on BL/SL with no peer.

Source of every value

lab-topology.DC1.yml and the DC1 lab configs rendered by Infrahub, main a7bcc19.
Layout drawn by hand; values copied, not typed.

DC1 lab path: addressing of every cable and device

The 43 cables (numbers match page 1; order as in lab-topology.DC1.yml)

#	End A (port)	End B (port)	Type	Addressing / VLANs / purpose
1	DC1-BL1 **1/1**	DC1-S1 **1/41**	fabric /31	10.19.4.80/31 · S1 .80, BL1 .81
2	DC1-BL1 **1/2**	DC1-S2 **1/41**	fabric /31	10.19.4.172/31 · S2 .172, BL1 .173
3	DC1-BL1 **1/3**	DC1-BL2 **1/3**	vPC peer-link	vPC 102 peer-link Po1000 · trunk 2531,2599,3001-3004,3300,3900
4	DC1-BL1 **1/4**	DC1-BL2 **1/4**	vPC peer-link	vPC 102 peer-link Po1000
5	DC1-BL1 **1/5**	DC1-BL2 **1/5**	vPC peer-link	vPC 102 peer-link Po1000
6	DC1-BL1 **1/6**	DC1-BL2 **1/6**	vPC peer-link	vPC 102 peer-link Po1000
7	DC1-BL1 **1/7**	DC1-INETFW1 **port19**	LACP bundle member	Po11 / vPC 11 ↔ agg-inside · VLAN 3001 TRANSIT 10.19.16.8/29 · BL1 .9, BL2 .10, FW .11
8	DC1-BL1 **1/34**	DC1-BL2 **1/34**	vPC keepalive	vPC 102 keepalive · VRF VPC-KEEPALIVE 10.19.17.2/31 · BL1 .2, BL2 .3
9	DC1-BL2 **1/1**	DC1-S1 **1/42**	fabric /31	10.19.4.82/31 · S1 .82, BL2 .83
10	DC1-BL2 **1/2**	DC1-S2 **1/42**	fabric /31	10.19.4.174/31 · S2 .174, BL2 .175
11	DC1-BL2 **1/7**	DC1-INETFW1 **port20**	LACP bundle member	Po11 / vPC 11 ↔ agg-inside · VLAN 3001 (second member)
12	DC1-EDGE1 **1/1**	DC1-EDGE2 **1/1**	vPC peer-link	vPC 104 peer-link Po1000 · trunk 3200
13	DC1-EDGE1 **1/2**	DC1-EDGE2 **1/2**	vPC peer-link	vPC 104 peer-link Po1000
14	DC1-EDGE1 **1/3**	DC1-EDGE2 **1/3**	vPC peer-link	vPC 104 peer-link Po1000
15	DC1-EDGE1 **1/4**	DC1-EDGE2 **1/4**	vPC peer-link	vPC 104 peer-link Po1000
16	DC1-EDGE1 **1/5**	DC1-ISP1 **Gi0-0-0-0**	LACP bundle member	Po21 / vPC 21 ↔ BE21 · VLAN 3200 OUTSIDE 10.19.14.0/24 · ISP1 .2
17	DC1-EDGE1 **1/6**	DC1-ISP2 **Gi0-0-0-0**	LACP bundle member	Po22 / vPC 22 ↔ BE21 · VLAN 3200 · ISP2 .3
18	DC1-EDGE1 **1/7**	DC1-INETFW1 **port21**	LACP bundle member	Po11 / vPC 11 ↔ agg-outside · VLAN 3200 · FW .10
19	DC1-EDGE1 **1/34**	DC1-EDGE2 **1/34**	vPC keepalive	vPC 104 keepalive · 10.19.17.6/31 · EDGE1 .6, EDGE2 .7
20	DC1-EDGE2 **1/5**	DC1-ISP1 **Gi0-0-0-1**	LACP bundle member	Po21 / vPC 21 ↔ BE21 · VLAN 3200 (second member)
21	DC1-EDGE2 **1/6**	DC1-ISP2 **Gi0-0-0-1**	LACP bundle member	Po22 / vPC 22 ↔ BE21 · VLAN 3200 (second member)
22	DC1-EDGE2 **1/7**	DC1-INETFW1 **port22**	LACP bundle member	Po11 / vPC 11 ↔ agg-outside · VLAN 3200 (second member)
23	DC1-INTFW1 **port19**	DC1-SL1 **1/7**	LACP bundle member	agg-inside ↔ Po11 / vPC 11 · VLANs 3000, 3100, 3101, 3105, 3106
24	DC1-INTFW1 **port20**	DC1-SL2 **1/7**	LACP bundle member	agg-inside ↔ Po11 / vPC 11 (second member)
25	DC1-ISP1 **Gi0-0-0-2**	DC1-ISP2 **Gi0-0-0-2**	routed	ISP cross-link 10.19.15.0/31 · ISP1 .0, ISP2 .1 · iBGP AS65050
26	DC1-L11 **1/49**	DC1-S1 **1/21**	fabric /31	10.19.4.40/31 · S1 .40, L11 .41
27	DC1-L11 **1/50**	DC1-S1 **1/22**	fabric /31	10.19.4.42/31 · S1 .42, L11 .43
28	DC1-L11 **1/51**	DC1-S2 **1/21**	fabric /31	10.19.4.132/31 · S2 .132, L11 .133
29	DC1-L11 **1/52**	DC1-S2 **1/22**	fabric /31	10.19.4.134/31 · S2 .134, L11 .135
30	DC1-S1 **1/43**	DC1-SL1 **1/1**	fabric /31	10.19.4.84/31 · S1 .84, SL1 .85
31	DC1-S1 **1/44**	DC1-SL2 **1/1**	fabric /31	10.19.4.86/31 · S1 .86, SL2 .87
32	DC1-S2 **1/43**	DC1-SL1 **1/2**	fabric /31	10.19.4.176/31 · S2 .176, SL1 .177
33	DC1-S2 **1/44**	DC1-SL2 **1/2**	fabric /31	10.19.4.178/31 · S2 .178, SL2 .179
34	DC1-SL1 **1/3**	DC1-SL2 **1/3**	vPC peer-link	vPC 103 peer-link Po1000 · trunk 2501-2502,2522,2531,2599,3000,3100-3101,3105-3106,3900
35	DC1-SL1 **1/4**	DC1-SL2 **1/4**	vPC peer-link	vPC 103 peer-link Po1000
36	DC1-SL1 **1/5**	DC1-SL2 **1/5**	vPC peer-link	vPC 103 peer-link Po1000
37	DC1-SL1 **1/6**	DC1-SL2 **1/6**	vPC peer-link	vPC 103 peer-link Po1000
38	DC1-SL1 **1/34**	DC1-SL2 **1/34**	vPC keepalive	vPC 103 keepalive · 10.19.17.4/31 · SL1 .4, SL2 .5
39	DC1-ISP1 **Gi0-0-0-3**	DC1-ISP-A **isp**	routed	192.0.2.0/30 · ISP1 .2, carrier A .1 · eBGP 65050–64496
40	DC1-ISP-A **host**	DC1-HOST-ISP-A **eth1**	host	198.18.1.0/24 · carrier A .1, host .10
41	DC1-ISP2 **Gi0-0-0-3**	DC1-ISP-B **isp**	routed	192.0.2.4/30 · ISP2 .6, carrier B .5 · eBGP 65050–64497
42	DC1-ISP-B **host**	DC1-HOST-ISP-B **eth1**	host	198.18.2.0/24 · carrier B .1, host .10
43	DC1-L11 **1/1**	DC1-HOST-PROD **eth1**	host	access VLAN 1010 (L2 VNI 11010) · VRF PROD 10.16.10.0/24 · anycast gw .1, host .10

The 18 devices

Lab node	Stands in for (production)	Production model	Lab image / size	BGP AS	loopback0 / loopback1 (VTEP)	mgmt (vamd-dc1-mgmt)
DC1-S1	Spine DC1-S1	N9K-C9364D-GX2A	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.1	10.250.0.2
DC1-S2	Spine DC1-S2	N9K-C9364D-GX2A	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.2	10.250.0.3
DC1-L11	Server leaf DC1-L11 (single-homed)	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.13 / 10.19.1.11	10.250.0.14
DC1-BL1	Border leaf DC1-BL1	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.23 / 10.19.1.21, sec 10.19.2.2	10.250.0.24
DC1-BL2	Border leaf DC1-BL2	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.24 / 10.19.1.22, sec 10.19.2.2	10.250.0.25
DC1-SL1	Service leaf DC1-SL1	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.25 / 10.19.1.23, sec 10.19.2.3	10.250.0.26
DC1-SL2	Service leaf DC1-SL2	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	65100	10.19.0.26 / 10.19.1.24, sec 10.19.2.3	10.250.0.27
DC1-EDGE1	Edge switch DC1-EDGE1	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	none (L2)	-	10.250.0.30
DC1-EDGE2	Edge switch DC1-EDGE2	Nexus 9000	vrnetlab/cisco_n9kv:10.6.4.M-lite · 8 GB, 3 vCPU	none (L2)	-	10.250.0.31
DC1-INTFW1	Internal FW DC1-INTFW1	FortiGate	quay.io/frrouting/frr:10.6.0	65110	router-id 10.19.16.3	10.250.0.40
DC1-INETFW1	Internet FW DC1-INETFW1	FortiGate	quay.io/frrouting/frr:10.6.0	65140	router-id 10.19.16.11	10.250.0.38
DC1-ISP1	ISP router DC1-ISP1	Cisco 8201, IOS XR	quay.io/frrouting/frr:10.6.0	65050	10.19.15.17	10.250.0.36
DC1-ISP2	ISP router DC1-ISP2	Cisco 8201, IOS XR	quay.io/frrouting/frr:10.6.0	65050	10.19.15.18	10.250.0.37
DC1-ISP-A	Carrier A (not a DC1 device)	-	quay.io/frrouting/frr:10.6.0	64496	router-id 192.0.2.1	10.250.0.240
DC1-ISP-B	Carrier B (not a DC1 device)	-	quay.io/frrouting/frr:10.6.0	64497	router-id 192.0.2.5	10.250.0.241
DC1-HOST-PROD	A server in VRF PROD	-	alpine:3.22	-	-	10.250.0.245
DC1-HOST-ISP-A	A host on the internet	-	alpine:3.22	-	-	10.250.0.246
DC1-HOST-ISP-B	A host on the internet	-	alpine:3.22	-	-	10.250.0.247

Spine–leaf numbering rule: the spine takes the lower address of each /31 (pool DC1-FABRIC-P2P, 10.19.4.0/22). vPC backup underlay SVI over the peer-link: VLAN 3900, BL1 10.19.3.8 / BL2 10.19.3.9, SL1 10.19.3.10 / SL2 10.19.3.11 (/31, OSPF). The secondary loopback1 address (10.19.2.2 BL pair, 10.19.2.3 SL pair) is the vPC pair's shared anycast VTEP.

BGP sessions in the lab

Session	Who (address)	AS ↔ AS	Where / notes
iBGP EVPN (l2vpn evpn)	DC1-L11, BL1, BL2, SL1, SL2 → S1 10.19.0.1 and S2 10.19.0.2	65100 ↔ 65100	loopback0 to loopback0, spines are route reflectors; BFD multihop 1000/1000/3 configured (not up on the virtual N9Kv)
eBGP VRF TRANSIT	SL1 .1 / SL2 .2 ↔ INTFW1 .3	65100 ↔ 65110	VLAN 3000, 10.19.16.0/29; FW vrf1
eBGP VRF PROD	SL1 .1 / SL2 .2 ↔ INTFW1 .3	65100 ↔ 65110	VLAN 3100, 10.19.8.0/29; FW vrf10
eBGP VRF DEV	SL1 .9 / SL2 .10 ↔ INTFW1 .11	65100 ↔ 65110	VLAN 3101, 10.19.8.8/29; FW vrf11
eBGP VRF CITRIX	SL1 .41 / SL2 .42 ↔ INTFW1 .43	65100 ↔ 65110	VLAN 3105, 10.19.8.40/29; FW vrf15
eBGP VRF PARTNER	SL1 .49 / SL2 .50 ↔ INTFW1 .51	65100 ↔ 65110	VLAN 3106, 10.19.8.48/29; FW vrf16
eBGP VRF TRANSIT	BL1 .9 / BL2 .10 ↔ INETFW1 .11	65100 ↔ 65140	VLAN 3001, 10.19.16.8/29; FW vrf1
eBGP OUTSIDE	INETFW1 .10 ↔ ISP1 .2 / ISP2 .3	65140 ↔ 65050	VLAN 3200 through EDGE1/EDGE2, 10.19.14.0/24
iBGP ISP pair	ISP1 10.19.15.0 ↔ ISP2 10.19.15.1	65050 ↔ 65050	cross-link Gi0-0-0-2, next-hop-self
eBGP carrier A	ISP1 192.0.2.2 ↔ carrier A 192.0.2.1	65050 ↔ 64496	ISP1 sends only 198.51.100.0/24
eBGP carrier B	ISP2 192.0.2.6 ↔ carrier B 192.0.2.5	65050 ↔ 64497	ISP2 sends only 198.51.100.0/24

Underlay: OSPF process UNDERLAY, area 0.0.0.0, point-to-point on every fabric /31, loopbacks advertised. BGP password on every session: `cisco123` (a public lab constant in the templates). The Internet FW announces 198.51.100.0/24 and sends 0/0 into TRANSIT; the Internal FW leaks TRANSIT 0/0 into each tenant VRF (route-maps RM-LEAK-TRANSIT-TO-PROD/-DEV/-CITRIX/-PARTNER) and the tenant summaries into TRANSIT (RM-LEAK-TO-TRANSIT).

VRFs (tenants)

VRF	L3 VNI	L3VNI VLAN	On (lab)	FW leg (VLAN → FW VRF)	Notes
PROD	50001	2501	L11, SL1, SL2	3100 → INTFW1 vrf10	L11: VLANs 1010–1017, 1040–1051 (10.16.x.1/24); host on VLAN 1010 / L2 VNI 11010
DEV	50002	2502	L11, SL1, SL2	3101 → INTFW1 vrf11	L11: VLANs 1110–1118, 1140–1150 (10.16.74–114.1/24); no host in the lab
CITRIX	50022	2522	L11, SL1, SL2	3105 → INTFW1 vrf15	L11: VLANs 1500–1501 (10.16.144–145.1/24); no host in the lab
PARTNER	50031	2531	SL1, SL2, BL1, BL2	3106 → INTFW1 vrf16	BL: VLAN 3300 to the VPN FWs (10.19.9.0/29, AS65130), not in the lab
TRANSIT	50100	2599	SL1, SL2, BL1, BL2	3000 → INTFW1 vrf1; 3001 → INETFW1	BL: VLANs 3002–3004 to SD-WAN FW and cloud routers, not in the lab

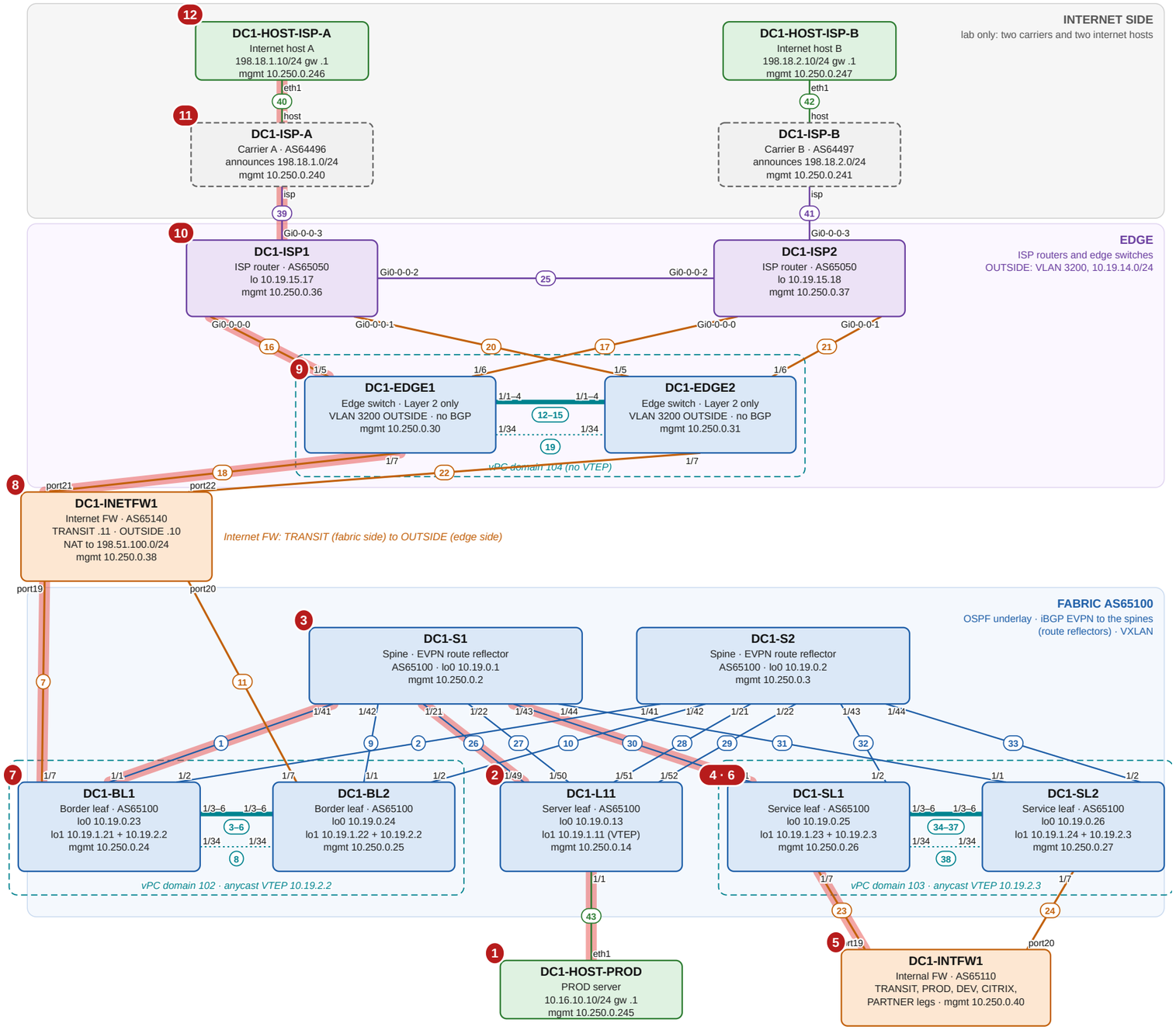
What a healthy lab shows (last full run, AWS 8 Oct evening)

`lab/postcheck.py DC1`: 108 of 164 checks pass. The 56 that fail are expected in this lab: 44 BFD checks (the virtual N9Kv does not run BFD) and 12 route checks for the DEV and CITRIX summaries (no host in those VRFs, so no aggregate). The PROD host reaches both internet hosts: 5 of 5 pings.

Firewall stand-ins: what they mirror and what they do not

The FRR stand-ins copy the rendered FortiOS / IOS XR routing (interfaces, VLANs, VRFs, BGP neighbours, prefix-lists, route-maps, statics). They do not run firewall policy, IPsec, the mgmt VRF or VRRP. The production FortiOS and IOS XR configs are rendered and checked, never run.

DC1 lab path: packet walk, the path of one packet from the PROD server to the internet



Return traffic takes the same path back: carrier A → ISP1 (198.51.100.0/24 is routed to INETFW1 10.19.14.10) → EDGE → INETFW1 (reverse NAT) → BL → EVPN TRANSIT → SL → INTFW1 → SL → EVPN PROD → L11 → host. With BL2, SL2, EDGE2 or ISP2 the packet can take the twin device of the vPC or ISP pair; the red line shows one of the equal paths.

the cables the packet crosses · **4 · 6** step numbers on the device (SL1 is crossed twice: once in VRF PROD, once in VRF TRANSIT) · colours and numbers as on page 1

1. **DC1-HOST-PROD (10.16.10.10/24) sends to its default gateway 10.16.10.1 out of eth1.**
   L11 Ethernet1/1 is an access port in VLAN 1010; VLAN 1010's SVI is the anycast gateway 10.16.10.1 in VRF PROD (same address and MAC 0200.0000.00aa on every leaf).

2. **DC1-L11 looks up 198.18.1.10 in VRF PROD and finds 0.0.0.0/0 learned through EVPN (route type 5) from the service leaves.**
   Check: `show ip route 0.0.0.0/0 vrf PROD` on DC1-L11. Captured: via 10.19.1.23 (SL1's own VTEP), segid 50001, tag 65110.

3. **L11 wraps the packet in VXLAN with L3 VNI 50001 (VRF PROD), outer source 10.19.1.11, outer destination 10.19.1.23; the spines route the outer packet over the OSPF underlay.**
   The spines never see the tenant packet: they only route between loopbacks. Check: `show nve peers` on L11.

4. **SL1 or SL2 removes the VXLAN header, looks up VRF PROD and sends to the Internal FW 10.19.8.3 on VLAN 3100 (port-channel 11).**
   Check: `show bgp vrf PROD ipv4 unicast summary` on SL1/SL2: neighbour 10.19.8.3, AS 65110, Established.

5. **DC1-INTFW1 receives in vrf10 (PROD). The 0/0 there was leaked from vrf1 (TRANSIT); it forwards into vrf1 and back to the SL pair on VLAN 3000 (10.19.16.1 / .2).**
   This is where production policy would inspect PROD → internet. Check: `docker exec -it clab-vamd-dc1-p3-DC1-INTFW1 vtysh -c 'show ip route vrf vrf10'`.

6. **SL1/SL2 look up VRF TRANSIT and find 0/0 from the border leaves through EVPN, L3 VNI 50100.**
   Check: `show ip route 0.0.0.0/0 vrf TRANSIT` on SL1. Captured: via 10.19.1.21 (BL1's own VTEP), segid 50100, tag 65140.

7. **BL1 or BL2 removes the VXLAN header and sends to the Internet FW 10.19.16.11 on VLAN 3001 (port-channel 11).**
   Check: `show bgp vrf TRANSIT ipv4 unicast summary` on BL1/BL2: neighbour 10.19.16.11, AS 65140.

8. **DC1-INETFW1 source-NATs the private address to one address of 198.51.100.1–126 and sends out of OUTSIDE (10.19.14.10, VLAN 3200, agg-outside).**
   The NAT is a lab stand-in for the FortiGate policy (iptables SNAT in the stand-in script); it worked in the AWS run of 8 Oct evening. Headers on every cable: page 5.

9. **EDGE1 / EDGE2 switch VLAN 3200 at Layer 2 to the ISP routers (port-channels 21 and 22). No routing, no BGP on the edge switches.**
   Check: `show vpc brief` and `show port-channel summary` on EDGE1.

10. **DC1-ISP1 (BE21.3200 10.19.14.2) has 198.18.1.0/24 from carrier A (eBGP 65050 ↔ 64496) and sends out of Gi0-0-0-3 to 192.0.2.1. ISP2 is the backup through carrier B.**
    Check: `docker exec -it clab-vamd-dc1-p3-DC1-ISP1 vtysh -c 'show bgp ipv4 unicast summary'`.

11. **Carrier A delivers to 198.18.1.0/24 on its host port.**
    Carrier A only exists in the lab; it plays the internet.

12. **DC1-HOST-ISP-A (198.18.1.10) replies to the public 198.51.100.x address.**
    Test from the PROD host: `docker exec -it clab-vamd-dc1-p3-DC1-HOST-PROD ping -c 5 198.18.1.10` → expected 5 of 5 (last run: 5 of 5).

DC1 lab path: packet walk, the headers on every cable

Outer Ethernet + IP	UDP 4789 + VXLAN	Inner Ethernet	Inner IP	payload
VTEP → VTEP (underlay)	VNI = which VRF / VLAN	router MAC → router MAC (routed)	host → internet host	

A packet between two VTEPs (L11 ↔ SL1, SL1 ↔ BL1). On every other cable only the inner part exists (with a VLAN tag on FW and edge links).

Router MAC (show nve peers)	Value
DC1-L11	`0c25.e000.1b08`
DC1-BL1	`0cac.c200.1b08`
DC1-BL2	`0cbb.8d00.1b08`
DC1-SL1	`0c9d.9700.1b08`
DC1-SL2	`0c4f.6f00.1b08`
BL pair (10.19.2.2)	`0200.0a13.0202`
SL pair (10.19.2.3)	`0200.0a13.0203`

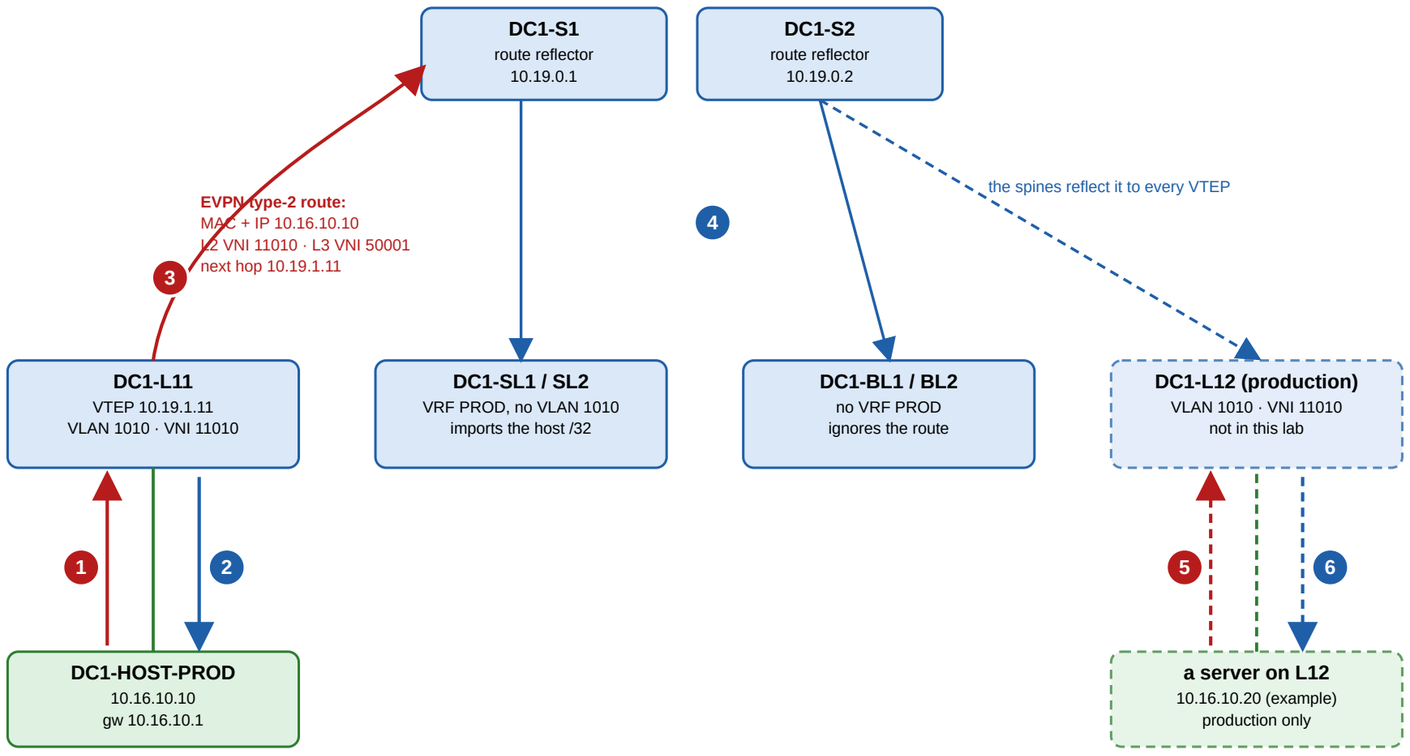
Cable (page 1 number)	802.1Q VLAN	Outer IP (VTEP → VTEP)	UDP / VXLAN	Inner Ethernet (src → dst MAC)	Inner IP (src → dst)	What the sending device did
HOST-PROD → L11 cable 43	none (access VLAN 1010 on L11)	no VXLAN on this cable		host eth1 MAC → 0200.0000.00aa (anycast gateway MAC)	10.16.10.10 → 198.18.1.10	The host sends to its gateway's MAC.
L11 → S1 cable 26 (or 27–29, ECMP)	none (routed /31)	10.19.1.11 (L11 VTEP) → 10.19.1.23 (SL1 VTEP)	UDP 4789 VXLAN VNI **50001 (L3 VNI PROD)**	0c25.e000.1b08 (L11 router MAC) → 0c9d.9700.1b08 (SL1 router MAC)	10.16.10.10 → 198.18.1.10	L11 routes in VRF PROD: 0/0 via 10.19.1.23, VNI 50001. The inner MACs become router MACs; a VXLAN header is added.
S1 → SL1 cable 30	none (routed /31)	10.19.1.11 → 10.19.1.23	UDP 4789 VXLAN VNI **50001**	0c25.e000.1b08 → 0c9d.9700.1b08	10.16.10.10 → 198.18.1.10	The spine routes only the outer IP (OSPF). Outer MACs change per hop; the inner packet is untouched.
SL1 → INTFW1 cable 23	3100 (PROD-INTFW)	no VXLAN on this cable		SL1 VLAN 3100 MAC → INTFW1 PROD-3100 MAC	10.16.10.10 → 198.18.1.10	SL1 removes VXLAN, routes in VRF PROD: 0/0 via 10.19.8.3 (eBGP AS65110).
INTFW1 → SL1 cable 23	3000 (TRANSIT-INTFW)	no VXLAN on this cable		INTFW1 TRANSIT-3000 MAC → SL VLAN 3000 MAC	10.16.10.10 → 198.18.1.10	The FW moves the packet from vrf10 (PROD) to vrf1 (TRANSIT) and sends it back to the SL pair.
SL1 → S1 cable 30	none (routed /31)	10.19.1.23 (SL1 VTEP) → 10.19.1.21 (BL1 VTEP)	UDP 4789 VXLAN VNI **50100 (L3 VNI TRANSIT)**	0c9d.9700.1b08 (SL1 router MAC) → 0cac.c200.1b08 (BL1 router MAC)	10.16.10.10 → 198.18.1.10	SL1 routes in VRF TRANSIT: 0/0 via 10.19.1.21, VNI 50100.
S1 → BL1 cable 1	none (routed /31)	10.19.1.23 → 10.19.1.21	UDP 4789 VXLAN VNI **50100**	0c9d.9700.1b08 → 0cac.c200.1b08	10.16.10.10 → 198.18.1.10	Spine: outer IP only.
BL1 → INETFW1 cable 7	3001 (TRANSIT-INETFW)	no VXLAN on this cable		BL1 VLAN 3001 MAC → INETFW1 TRANSIT-3001 MAC	10.16.10.10 → 198.18.1.10	BL1 removes VXLAN, routes in TRANSIT: 0/0 via 10.19.16.11 (eBGP AS65140).
INETFW1 → EDGE1 → ISP1 cable 18, then 16	3200 (OUTSIDE)	no VXLAN on this cable		INETFW1 OUTSIDE-3200 MAC → ISP1 BE21.3200 MAC	198.51.100.x (NAT, from .1–.126) → 198.18.1.10	Source NAT: the private address becomes one address of 198.51.100.1–126. EDGE1 switches VLAN 3200 at Layer 2 only.
ISP1 → carrier A cable 39	none (routed /30)	no VXLAN on this cable		ISP1 Gi0-0-0-3 MAC → carrier A isp port MAC	198.51.100.x → 198.18.1.10	ISP1 routes 198.18.1.0/24 learned from carrier A (eBGP 65050 ↔ 64496).
carrier A → HOST-ISP-A cable 40	none	no VXLAN on this cable		carrier A host port MAC → host A eth1 MAC	198.51.100.x → 198.18.1.10	Delivered.

Where the values come from. VTEP addresses, VNIs, VLANs and the anycast MAC: the rendered configs. Router MACs and the next hops (L11 VRF PROD 0/0 via 10.19.1.23 VNI 50001; SL1 VRF PROD 0/0 via 10.19.8.3; SL1 TRANSIT 0/0 via 10.19.1.21 VNI 50100; BL1 TRANSIT 0/0 via 10.19.16.11): AWS run, 9 Oct 2026 01:53 UTC (captures t10/t11 'before'). NAT range: the Internet FW stand-in script (`SNAT --to-source 198.51.100.1-198.51.100.126`). Fields in plain words ("INTFW1 PROD-3100 MAC") were not captured. Router MACs of a virtual Nexus are new on every deploy; the anycast gateway MAC 0200.0000.00aa and the virtual router MACs 0200.0a13.0202/0203 (built from the anycast VTEP address) stay.

Why VTEP 10.19.1.23 and not the SL pair's 10.19.2.3. The pairs run `advertise-pip`: routes a vPC switch learns itself (here the FW's 0/0) are sent with the switch's own VTEP address and router MAC, so traffic goes to the switch that has the route. Host MACs behind a vPC pair use the shared address 10.19.2.3 / 10.19.2.2 with the virtual router MAC. At capture time L11 had one path (via SL1).

Return. Carrier A → ISP1, which routes 198.51.100.0/25 to the Internet FW 10.19.14.10 (static) → EDGE (VLAN 3200) → INETFW1 reverses the NAT → BL (TRANSIT; tenant summary learned from the SL pair through EVPN) → SL → INTFW1 → SL (VRF PROD; the host route 10.16.10.10/32 is an EVPN type-2 from L11) → L11 → host.

DC1 lab path: the first ARP and how the fabric learns a host



Config lines behind it (every leaf)

```
fabric forwarding anycast-gateway-mac 0200.0000.00aa
interface Vlan1010
  fabric forwarding mode anycast-gateway
interface nve1
  global suppress-arp
  global ingress-replication protocol bgp
  member vni 11010
```

ARP suppression needs the TCAM region the `carve` step sets on the virtual Nexus (`arp-ether 256 double-wide`). Job 388 found `suppress-arp` in L11's running-config after the carve.

Dashed boxes and arrows: production behaviour that this lab cannot show. Option 5 (entire DC1) has all 20 leaves; it needs a second test host to show steps 5–6.

1 ARP request (broadcast): who has 10.16.10.1? **2** L11 answers itself: 10.16.10.1 is at 0200.0000.00aa (anycast gateway, on every leaf)

5 the L12 server asks for 10.16.10.10 **6** L12 answers from its EVPN table with the host's MAC (ARP suppression): the request never crosses the fabric

Step	What happens	Seen in the lab?	Look at it
1	HOST-PROD sends ARP request *who has 10.16.10.1* (broadcast, VLAN 1010).	Seen in the lab: the host pings its gateway (lab-expected ping 1).	`docker exec -it clab-vamd-dc1-p3-DC1-HOST-PROD ip neigh`
2	L11 owns 10.16.10.1 (anycast gateway, `fabric forwarding mode anycast-gateway`) and replies with MAC 0200.0000.00aa. Nothing crosses the fabric.	Lab ✓	`show ip arp vrf PROD` on L11
3	L11 learns the host: MAC in VLAN 1010 on Ethernet1/1, IP → MAC in VRF PROD. It sends one EVPN route type 2 (MAC + IP, L2 VNI 11010, L3 VNI 50001, router MAC, next hop 10.19.1.11) to both spines.	Lab ✓: the return path to the host works (5 of 5 pings), and it needs this route	`show mac address-table vlan 1010`, `show bgp l2vpn evpn 10.16.10.10`
4	The spines reflect it to every VTEP. SL1/SL2 have VRF PROD but no VLAN 1010: they import the host /32 into VRF PROD (that is how return traffic finds L11). BL1/BL2 have no VRF PROD and drop it.	Lab ✓ for SL (return path works)	`show ip route 10.16.10.10 vrf PROD` on SL1
5–6	A server on another leaf with VLAN 1010 (production: L1–L20) asks for 10.16.10.10. Its leaf answers from the EVPN entry (`suppress-arp`): no flood.	Production only: the lab path has one server leaf	`show ip arp suppression-cache detail`
–	An address nobody has learned yet: the ARP request is copied to every VTEP that has VNI 11010 (ingress replication, list built from EVPN type-3 routes).	In the lab the list is empty: L11 is the only VTEP with VNI 11010	`show nve vni 11010`, `show l2route evpn imet all`

DC1 lab path: lab cables vs production cables, and what is configured but missing

Each lab cable is a real DC1 cable (from the vendor cable schedule rendered by Infrahub)

#	Lab cable (containerlab names)	Production cable label	Media	Length	Route (patch panel ports)
1	BL1 **eth1** ↔ S1 **eth41**	DC1-S1:Eth1/41-DC1-BL1:Eth1/1	dac	2 m	same rack DC1-B04
2	BL1 **eth2** ↔ S2 **eth41**	DC1-S2:Eth1/41-DC1-BL1:Eth1/2	smf	26 m	DC1-B08-PP42:83 → DC1-B04-PP42:83
3	BL1 **eth3** ↔ BL2 **eth3**	DC1-BL1:Eth1/3-DC1-BL2:Eth1/3	smf	26 m	DC1-B04-PP42:1 → DC1-B08-PP42:1
4	BL1 **eth4** ↔ BL2 **eth4**	DC1-BL1:Eth1/4-DC1-BL2:Eth1/4	smf	26 m	PP42:2 → PP42:2
5	BL1 **eth5** ↔ BL2 **eth5**	DC1-BL1:Eth1/5-DC1-BL2:Eth1/5	smf	26 m	PP42:3 → PP42:3
6	BL1 **eth6** ↔ BL2 **eth6**	DC1-BL1:Eth1/6-DC1-BL2:Eth1/6	smf	26 m	PP42:4 → PP42:4
7	BL1 **eth7** ↔ INETFW1 **port19**	DC1-BL1:Eth1/7-DC1-INETFW1:port19	dac	3 m	same rack DC1-B04
8	BL1 **eth34** ↔ BL2 **eth34**	DC1-BL1:Eth1/34-DC1-BL2:Eth1/34	smf	26 m	PP42:9 → PP42:9
9	BL2 **eth1** ↔ S1 **eth42**	DC1-S1:Eth1/42-DC1-BL2:Eth1/1	smf	26 m	DC1-B04-PP42:80 → DC1-B08-PP42:40
10	BL2 **eth2** ↔ S2 **eth42**	DC1-S2:Eth1/42-DC1-BL2:Eth1/2	dac	2 m	same rack DC1-B08
11	BL2 **eth7** ↔ INETFW1 **port20**	DC1-BL2:Eth1/7-DC1-INETFW1:port20	smf	27 m	DC1-B08-PP42:10 → DC1-B04-PP42:10
12	EDGE1 **eth1** ↔ EDGE2 **eth1**	DC1-EDGE1:Eth1/1-DC1-EDGE2:Eth1/1	smf	26 m	PP42:22 → PP42:22
13	EDGE1 **eth2** ↔ EDGE2 **eth2**	DC1-EDGE1:Eth1/2-DC1-EDGE2:Eth1/2	smf	26 m	PP42:23 → PP42:23
14	EDGE1 **eth3** ↔ EDGE2 **eth3**	DC1-EDGE1:Eth1/3-DC1-EDGE2:Eth1/3	smf	26 m	PP42:24 → PP42:24
15	EDGE1 **eth4** ↔ EDGE2 **eth4**	DC1-EDGE1:Eth1/4-DC1-EDGE2:Eth1/4	smf	26 m	PP42:25 → PP42:25
16	EDGE1 **eth5** ↔ ISP1 **Gi0-0-0-0**	DC1-EDGE1:Eth1/5-DC1-ISP1:FH0/0/0/0	dac	2 m	same rack DC1-B04
17	EDGE1 **eth6** ↔ ISP2 **Gi0-0-0-0**	DC1-EDGE1:Eth1/6-DC1-ISP2:FH0/0/0/0	smf	26 m	DC1-B04-PP42:26 → DC1-B08-PP42:26
18	EDGE1 **eth7** ↔ INETFW1 **port21**	DC1-EDGE1:Eth1/7-DC1-INETFW1:port21	dac	3 m	same rack DC1-B04
19	EDGE1 **eth34** ↔ EDGE2 **eth34**	DC1-EDGE1:Eth1/34-DC1-EDGE2:Eth1/34	smf	26 m	PP42:30 → PP42:30
20	EDGE2 **eth5** ↔ ISP1 **Gi0-0-0-1**	DC1-EDGE2:Eth1/5-DC1-ISP1:FH0/0/0/2	smf	26 m	DC1-B08-PP42:31 → DC1-B04-PP42:31
21	EDGE2 **eth6** ↔ ISP2 **Gi0-0-0-1**	DC1-EDGE2:Eth1/6-DC1-ISP2:FH0/0/0/2	dac	2 m	same rack DC1-B08
22	EDGE2 **eth7** ↔ INETFW1 **port22**	DC1-EDGE2:Eth1/7-DC1-INETFW1:port22	smf	27 m	DC1-B08-PP42:32 → DC1-B04-PP42:32
23	INTFW1 **port19** ↔ SL1 **eth7**	DC1-SL1:Eth1/7-DC1-INTFW1:port19	dac	3 m	same rack DC1-B04
24	INTFW1 **port20** ↔ SL2 **eth7**	DC1-SL2:Eth1/7-DC1-INTFW1:port20	smf	27 m	DC1-B08-PP42:94 → DC1-B04-PP42:94
25	ISP1 **Gi0-0-0-2** ↔ ISP2 **Gi0-0-0-2**	DC1-ISP1:FH0/0/0/4-DC1-ISP2:FH0/0/0/4	smf	26 m	DC1-B04-PP42:39 → DC1-B08-PP42:39
26	L11 **eth49** ↔ S1 **eth21**	DC1-S1:Eth1/21-DC1-L11:Eth1/49	smf	28 m	DC1-B04-PP42:60 → DC1-C01-PP42:1
27	L11 **eth50** ↔ S1 **eth22**	DC1-S1:Eth1/22-DC1-L11:Eth1/50	smf	28 m	DC1-B04-PP42:61 → DC1-C01-PP42:2
28	L11 **eth51** ↔ S2 **eth21**	DC1-S2:Eth1/21-DC1-L11:Eth1/51	smf	31 m	DC1-B08-PP42:63 → DC1-C01-PP42:3
29	L11 **eth52** ↔ S2 **eth22**	DC1-S2:Eth1/22-DC1-L11:Eth1/52	smf	31 m	DC1-B08-PP42:64 → DC1-C01-PP42:4
30	S1 **eth43** ↔ SL1 **eth1**	DC1-S1:Eth1/43-DC1-SL1:Eth1/1	dac	2 m	same rack DC1-B04
31	S1 **eth44** ↔ SL2 **eth1**	DC1-S1:Eth1/44-DC1-SL2:Eth1/1	smf	26 m	DC1-B04-PP42:81 → DC1-B08-PP42:41
32	S2 **eth43** ↔ SL1 **eth2**	DC1-S2:Eth1/43-DC1-SL1:Eth1/2	smf	26 m	DC1-B08-PP42:84 → DC1-B04-PP42:84
33	S2 **eth44** ↔ SL2 **eth2**	DC1-S2:Eth1/44-DC1-SL2:Eth1/2	dac	2 m	same rack DC1-B08
34	SL1 **eth3** ↔ SL2 **eth3**	DC1-SL1:Eth1/3-DC1-SL2:Eth1/3	smf	26 m	PP42:88 → PP42:88
35	SL1 **eth4** ↔ SL2 **eth4**	DC1-SL1:Eth1/4-DC1-SL2:Eth1/4	smf	26 m	PP42:89 → PP42:89
36	SL1 **eth5** ↔ SL2 **eth5**	DC1-SL1:Eth1/5-DC1-SL2:Eth1/5	smf	26 m	PP42:90 → PP42:90
37	SL1 **eth6** ↔ SL2 **eth6**	DC1-SL1:Eth1/6-DC1-SL2:Eth1/6	smf	26 m	PP42:91 → PP42:91
38	SL1 **eth34** ↔ SL2 **eth34**	DC1-SL1:Eth1/34-DC1-SL2:Eth1/34	smf	26 m	PP42:93 → PP42:93
39	ISP1 **Gi0-0-0-3** ↔ ISP-A **isp**	DC1-ISP1:FH0/0/0/6 carrier handoff (ISP-A)	carrier	-	outside the site
40	ISP-A **host** ↔ HOST-ISP-A **eth1**	lab only	-	-	-
41	ISP2 **Gi0-0-0-3** ↔ ISP-B **isp**	DC1-ISP2:FH0/0/0/6 carrier handoff (ISP-B)	carrier	-	outside the site
42	ISP-B **host** ↔ HOST-ISP-B **eth1**	lab only	-	-	-
43	L11 **eth1** ↔ HOST-PROD **eth1**	DC1-L11:Eth1/1 lab test-host port (added for the lab, P3-39)	-	-	server port

Port names: containerlab eth*N* = Ethernet1/*N* on the Nexus; the ISP stand-ins' Gi0-0-0-*k* = FourHundredGigE0/0/0/*2k* on the Cisco 8201 (0 → FH0/0/0/0, 1 → FH0/0/0/2, 2 → FH0/0/0/4 cross-link, 3 → FH0/0/0/6 carrier). Same-rack cables are DAC; cross-rack cables are single-mode fibre through the patch racks (length = jumpers + trunks). Cable 43 is the lab's only added port.

Configured on a lab device, but the far end is not in the lab (expected to be down)

Lab device	Ports with no far end	BGP sessions that stay down
DC1-S1 / S2	Eth1/1–1/20, 1/23–1/40 (L1–L10, L12–L20), Eth1/45–46 (BGW1/2)	21 of 26 EVPN route-reflector clients (only L11, BL1/2, SL1/2 are here)
DC1-BL1 / BL2	Eth1/8 INETFW2, 1/9–1/10 SDWANFW1/2, 1/11–1/12 VPNFW1/2, 1/13–1/14 CLOUD1/2	TRANSIT: 10.19.16.19 (SD-WAN FW AS65120), 10.19.16.27 / .35 (cloud routers AS65170); PARTNER: 10.19.9.3 (VPN FW AS65130)
DC1-SL1 / SL2	Eth1/8 INTFW2 (port-channel 12)	none
DC1-EDGE1 / EDGE2	Eth1/8 INETFW2, 1/9–1/12 SDWANFW1/2 and VPNFW1/2	none (Layer 2 only)
DC1-INETFW1	port15–16 agg-dmz (to DMZ1/DMZ2, VLANs 1400–1415)	none; the DMZ subnets 10.16.128–143.0/24 exist on the FW with no host
DC1-ISP1 / ISP2	none in the lab path	VRRP not run on FRR
DC1-L11	none (single-homed leaf)	none

These are the reason `show interface status`, `show vpc brief` and `show bgp … summary` show ports, vPCs and sessions down in a healthy lab. `postcheck.py` checks only the sessions in `lab-expected`, so they do not count as failures.

VRFs and their segments

VRF	L3 VNI	L3VNI VLAN	Tenant VLANs	L2 VNIs	Subnets (gateway .1)	On lab devices	FW leg
PROD	50001	2501	1010–1017, 1040–1051	11010–11017, 11040–11051	10.16.10–17.0/24, 10.16.40–51.0/24	L11; SL1/2 (L3 only)	3100 → INTFW1 vrf10
DEV	50002	2502	1110–1118, 1140–1150	11110–11118, 11140–11150	10.16.74–82.0/24, 10.16.104–114.0/24	L11; SL1/2 (L3 only)	3101 → INTFW1 vrf11
CITRIX	50022	2522	1500–1501	11500–11501	10.16.144–145.0/24	L11; SL1/2 (L3 only)	3105 → INTFW1 vrf15
PARTNER	50031	2531	–	–	–	SL1/2, BL1/2 (L3 only)	3106 → INTFW1 vrf16; BL 3300 → VPN FW (not in lab)
TRANSIT	50100	2599	–	–	–	SL1/2, BL1/2 (L3 only)	3000 → INTFW1 vrf1; BL 3001 → INETFW1
REPLICATION	50041	2541	1598	11598	10.16.176.0/24	not on any lab device (BGWs)	–

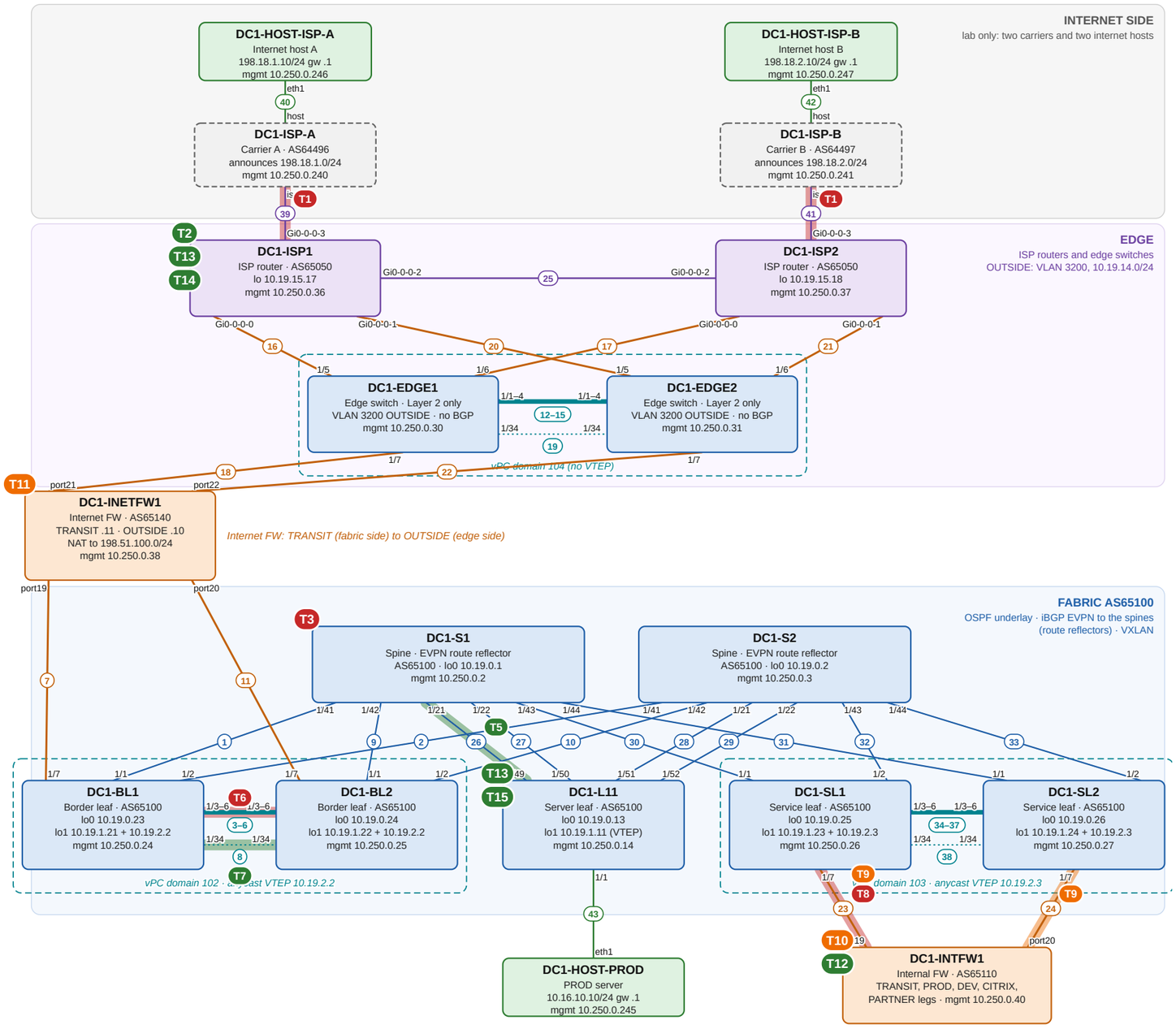
Rule seen in the data: L2 VNI = 10000 + VLAN. Every tenant subnet's gateway is the anycast address .1 with MAC 0200.0000.00aa on every leaf that has the VLAN. The PROD host sits in VLAN 1010 (10.16.10.0/24). L3-only means the VRF exists with its L3 VNI but no tenant VLAN (the switch routes for it, hosts are elsewhere).

VTEPs in the lab

VTEP	Own address (loopback1)	Shared vPC address (secondary)	Router MAC (AWS run, 9 Oct 2026 01:53 UTC (captures t10/t11 'before'))	VNIs it carries
DC1-L11	10.19.1.11	–	0c25.e000.1b08	PROD, DEV, CITRIX + 44 L2 VNIs
DC1-SL1	10.19.1.23	10.19.2.3	0c9d.9700.1b08	PROD, DEV, CITRIX, PARTNER, TRANSIT
DC1-SL2	10.19.1.24	10.19.2.3	0c4f.6f00.1b08	same as SL1
DC1-BL1	10.19.1.21	10.19.2.2	0cac.c200.1b08	PARTNER, TRANSIT
DC1-BL2	10.19.1.22	10.19.2.2	0cbb.8d00.1b08	same as BL1

Shared addresses carry the virtual router MACs 0200.0a13.0203 (SL pair) and 0200.0a13.0202 (BL pair). BUM traffic: ingress replication over BGP (EVPN type-3), no multicast in the underlay. Check: `show nve vni`, `show nve peers`, `show bgp l2vpn evpn summary` (type-2 / type-5 counts).

DC1 lab path: where each failure test breaks the lab, and its last result



#	Test	Breaks	Last result	Note
1	Carrier loss: carrier A, then both carriers	cables 39, 41	FAIL (1 check)	conditional default still sent 45 s after both carriers dropped (graceful-restart stale path, not confirmed)
2	ISP router down: DC1-ISP1	DC1-ISP1	PASS 0.2 s	
3	Spine down: DC1-S1	DC1-S1	FAIL 39.2 s	limit 5 s; no BFD on the virtual Nexus (platform)
5	One leaf–spine link down: L11 eth49	cable 26	PASS 0.1 s	
6	vPC peer-link down (BL pair)	cables 3–6	FAIL (1 check)	3-ping check failed at 45 s; continuous ping recovered (accepted for go-live, not retested)
7	vPC keepalive down (BL1 eth34)	cable 8	PASS 0.2 s	
8	One Internal FW bundle member down	cable 23	FAIL	sessions stay up, traffic stops (accepted for go-live, not retested)
9	Silent failure toward the Internal FW (100 % loss)	cables 23, 24	PASS	BFD check blocked (platform)
10	Internal FW stand-in down	DC1-INTFW1	FAIL (method)	docker pause keeps kernel forwarding; use ip link down
11	Internet FW stand-in down	DC1-INETFW1	FAIL (method)	same as 10
12	Filter test: Internal FW announces 10.99.0.0/16	DC1-INTFW1	PASS	8 Oct night run
13	BGP graceful restart: L11, then ISP1	DC1-L11, DC1-ISP1	PASS 0.1 s	
14	ISP1 returns (VRRP preempt wait)	DC1-ISP1	PASS 0.2 s	
15	Node reload with its rendered config: L11	DC1-L11	PASS	8 Oct night run

Last full run: AWS, 8 Oct 2026 evening (tests 12 and 15 from the 8 Oct night run, by decision). Test 4 was removed (8 Oct). List them: `.venv-lab/bin/python lab/failtests.py DC1 --list`. Each test checks the sessions that were up just before the break, measures the outage from the break to the first reply (ping every 0.2 s) and runs the full post-check after the restore. Exact break and restore commands: the 8 Oct runbook (`lab-results/aws-20261008-eve`).

green pass · **red** fail · **orange** blocked by the platform or the test method. Badge on a device: the test stops or restarts that device. Badge on a cable: the test cuts that cable.

What you need

Item	Value
Memory	About 80 GB free for the lab: 9 Nexus × 8 GB (QEMU_MEMORY 8192) = 72 GB plus 9 FRR / Alpine containers. Ran on AWS m8i.8xlarge (32 vCPU, 123 GB).
CPU	Each Nexus 3 vCPU (QEMU_SMP 3); nested virtualization (KVM) on the host.
Software	containerlab 0.79, Docker, image `vrnetlab/cisco_n9kv:10.6.4.M-lite` (built from your own Cisco download), `quay.io/frrouting/frr:10.6.0`, `alpine:3.22`.
Infrahub	Running somewhere you can reach for `fetch` (laptop is fine); the lab host does not need it after fetch.
Time	27 minutes from `deploy` to a fully configured lab, no manual step.

Bring it up

```
# files from Infrahub, compared with the repo
.venv/bin/python lab/labctl.py fetch DC1 --compare-baselines
# topology, files, images, memory
.venv/bin/python lab/labctl.py validate DC1
# boots all 18 nodes (minimal Nexus config)
.venv/bin/python lab/labctl.py deploy DC1
# full rendered configs over NX-API
.venv/bin/python lab/labctl.py push DC1
# TCAM for ARP suppression, save, reload, re-push
.venv/bin/python lab/labctl.py carve DC1
# 164 checks; expect 108 to pass (page 3)
.venv-lab/bin/python lab/postcheck.py DC1
```

Take it down

```
.venv/bin/python lab/labctl.py destroy DC1
```

Log in

Device kind	How
Nexus (9)	`ssh vamdadmin@<mgmt address>` (page 1 / page 2), password from `ztp/make-secret.sh`
FRR stand-ins (6)	`docker exec -it clab-vamd-dc1-p3-DC1-INTFW1 vtysh` (any FRR node name)
Hosts (3)	`docker exec -it clab-vamd-dc1-p3-DC1-HOST-PROD sh`

Stop and start one device

The topology file containerlab uses is `lab/build/DC1/topology.boot.yml` (`topology.override.yml` when `deploy` ran with `--memory`, as on a laptop). Find it with `docker inspect -f '{{index .Config.Labels "clab-topo-file"}}' clab-vamd-dc1-p3-DC1-L11`.

Device kind	Stop	Start	Tested
FRR stand-ins, hosts	`containerlab stop -t <file> --node DC1-INTFW1`	`containerlab start -t <file> --node DC1-INTFW1`	**Laptop 19 Oct: back in seconds, config, VRFs, addresses and routes the same**
Nexus	`containerlab stop -t <file> --node DC1-L11` works	Per-node start did not bring the switch back (40 min). Use destroy + deploy, push, carve.	**Laptop 19 Oct: start failed**

Cut a cable without stopping anything

```
# Nexus side, logged in to the switch (example: L11 uplink to S1, cable 26)
configure terminal
interface Ethernet1/49
  shutdown            # "no shutdown" puts it back
# FRR side (example: Internal FW member to SL1, cable 23)
docker exec clab-vamd-dc1-p3-DC1-INTFW1 ip link set port19 down    # "up" puts it back
```

Smaller labs from the same files

Option	Nodes	Memory	Status
1 · one switch	DC1-L11 (`--node-filter DC1-L11 --memory 6144`)	about 7 GB free	Runs on the 12 GB laptop
2 · DC1 lab path (this document, 10 pages)	18	about 80 GB	Ran on AWS 8 Oct
3 · DC2 lab path	17	about 80 GB	Rendered, never run
4 · DC1 + DC2 lab paths	35 (two separate labs)	about 160 GB	Not run
5 · entire DC1 production	46 + stand-ins	about 290 GB	Being written